With the rise of heterogeneous systems, there is a requirement for a scalable software system that is cost-effective and maintenance-free. Virtual machines (VMs) provide abstraction from the heterogeneity, and present a low-cost, scalable software development environment, as in the case of VM-based modern programming languages like Java. In this article, we will cover the fundamental concepts of a VM, using one of the critical components of Google's Android OS—the Dalvik VM.

A virtual machine (or VM) is an isolated, self-contained implementation that abstracts hardware resources. Typically, it simulates an actual physical machine capable of executing instruction sets. Broadly, VMs are classified based on how they work and their functionality. There are process VMs, which support the execution of a single process/program (e.g., Java, .NET), and system VMs, which support execution of a complete operating system (e.g., VirtualBox, VMWare).

How does a VM work?

In general, a VM is an interpreter for an instruction-set. The fundamental task of a compute machine is to carry out an operation. An operation is performed with the execution of a set of instructions. A basic instruction goes through the following cycle of events:

- Instruction fetch, where an instruction is fetched from memory.
- Decoding, which determines the type of the instruction—the opcode, or the operation it is supposed to perform. Additionally, decoding may also involve fetching operand(s) from memory.
- Execute. The decoded instruction is then executed; the operation is performed on the operands.
- Storing the result in the mentioned register.

Which VM implementations to choose

Mainly, VMs are implemented in one of two ways—a register-based VM, or a stack-based VM. The register-based VM computation model assumes registers as the memory, and operands are accessed with explicit addresses coded in an instruction. In the stack-based VM model, memory takes the form of a Last-In-First-Out (LIFO) stack. Every operation boils down to a stack PUSH/POP or STORE/LOAD. These instructions do not need explicit operand addresses, because the operand is always available with the stack pointer (top-of-stack). Examples are Microsoft .NET and the Sun JVM.